

Databázové systémy

Jazyk SQL - Príkaz SELECT

Ing. Ján Perháč, PhD.

Katedra počítačov a informatiky
Fakulta elektrotechniky a informatiky
Technická univerzita v Košiciach



Košice, Slovensko

Príkaz SELECT

Základná syntax

- Najdôležitejší príkaz v SQL.
- Získa (vyfiltruje) údaje z tabuľky.
 - Vybrané riadky (WHERE).
 - Vybrané stĺpce (SELECT).
- Syntax:

```
SELECT   co
FROM    odkial
WHERE    co_nas_zaujima
ORDER BY podla_coho_triedit
DESC;
```

Príkaz SELECT

Príkaz SELECT - Príklad

```
SELECT * FROM student;
```

```
SELECT meno, priezvisko  
FROM student;
```

```
SELECT meno, priezvisko  
FROM student  
ORDER BY priezvisko, meno;
```

```
SELECT meno, priezvisko  
FROM student  
WHERE meno = 'Janko';
```

Základné (logické) operátory a funkcie

Základné operátory

- Relačné:

=, <, >, <>, !=, <=, >=

- Aritmetické:

+, -, *, /, MOD(a,b)

- Spojenie dvoch reťazcov (konkatenácia):

'string1' || 'string2'

- Test na hodnotu *NULL*:

IS [NOT] NULL

Výraz môže byť:

- V klauzule *WHERE* pri formulácii podmienky selekcie.
- V zozname stĺpcov projekcie príkazu *SELECT*.
- V klauzule *ORDER BY*.

Základné (logické) operátory a funkcie

Základné logické operátory

- **AND** - "a zároveň platí"
meno='Janko' AND priezvisko='Hrasko'
- **OR** - "alebo"
predmet='SQL' OR predmet='DBS'
predmet IN ('SQL', 'DBS')
- **NOT** - negácia
predmet NOT IN ('SQL', 'DBS')

Základné (logické) operátory a funkcie

Základné funkcie na reťazcoch

- *upper(string)* - prevod na veľké písmená.
- *lower(string)* - prevod na malé písmená.
- *substr(string, from [, count])* - podreťazec.
- *initcap(string)* - kapitálky.
- *replace(string text, from text, to text)* - náhrada reťazca.
- *length(string)* - dĺžka reťazca.
- *md5(string)* - vypočíta MD5 hash parametra.
- *ascii(string)* - ASCII kód prvého symbolu reťazca.
- *chr(int)* - ASCII kód na symbol.

Základné (logické) operátory a funkcie

Základné funkcie na reťazcoch - príklad

Príklad	Výsledok
upper('tom')	TOM
lower('TOM')	tom
substr('alphabet', 3, 2)	ph
initcap('hi THOMAS')	Hi Thomas
replace('abcdefabcdef', 'cd', 'XX')	abXXefabXXef
length('jose')	4
md5('gop')	1c6b57c90...
ascii('a')	97
chr(97)	a

Základné (logické) operátory a funkcie

Základné funkcie na dátume a čase

- *age(timestamp, timestamp)* - odčíta argumenty.
- *age(timestamp)* - odčíta argument od *current_date*.
- *current_date* - vráti aktuálny dátum.
- *current_time* - vráti aktuálny čas.
- *current_timestamp* - vráti aktuálny čas a dátum (ekvivalent *now()*).
- *extract(field FROM source)* - vráti napr. rok, hodinu, minútu a pod. zo *source*, pričom *source* musí byť typu *timestamp*, *time*, *interval*, *date*.

Základné (logické) operátory a funkcie

Základné funkcie na dátume a čase - príklad

Príklad	Výsledok
age(timestamp '1957-06-13')	63 years 8 mons 24 days ...
date '2001-10-01' - date '2001-09-28'	3
time '05:00' - time '03:00'	02:00:00
time '05:00' - interval '2 hours'	03:00:00
900 * interval '1 second'	00:15:00

- SELECT EXTRACT(CENTURY FROM TIMESTAMP '2000-12-16 12:21:13');
Result: 20
- SELECT EXTRACT(ISOYEAR FROM DATE '2006-01-02');
Result: 2006
- SELECT EXTRACT(MONTH FROM TIMESTAMP '2001-02-16 20:38:40');
Result: 2

Základné (logické) operátory a funkcie

Základné funkcie na číslach

- *abs(-5)* - absolútna hodnota.
- *ceil(15.7)* resp. *floor(15.7)* - najbližšie väčšie resp. menšie celé číslo.
- *mod(123,5)* - zvyšok po delení.
- *round(15.7)* - zaokrúhlenie.
- *sign(-5)* - znamienko.
- *trunc(165.54, 1)* - orezanie desatinných miest.
- *greatest(1,5,7,33)* - najväčší prvok.
- *least(1,5,7,33)* - najmenší prvok.
- *pi()* - konštanta π .

Klauzuly

SELECT DISTINCT

- Tabuľka v relačných DBS je multimnožina - umožňuje mať viacero rovnakých záznamov.
- **SELECT DISTINCT** - odstráni duplikáty vo výsledku.
- Príklad:

```
SELECT DISTINCT meno  
FROM student;
```

Klauzuly

LIKE

- LIKE - slúži na vyhľadávanie prostredníctvom vzorov:
 - % - 0 až N ľubovoľných znakov,
 - _ - 1 ľubovoľný znak.
- Príklad:

```
SELECT priezvisko , meno  
FROM student  
WHERE priezvisko LIKE '%ova'  
AND meno LIKE '_ana'
```

Klauzuly

BETWEEN

- Slúži na definovanie rozsahu hodnôt od-do (vrátane) alebo časových období (dátumov).
- Príklad:

```
SELECT priezvisko , meno  
FROM student  
WHERE rocnik BETWEEN 1 AND 5;
```

```
SELECT * FROM faktura  
WHERE datum_vystavenia  
NOT BETWEEN '2013-12-01' AND '2013-12-31';
```

Klauzuly

Triedenie záznamov

- Triedenie podľa hodnôt atribútov:
 - vzostupne (ASC - predvolené),
 - zostupne (DESC).
- Príklad:

```
SELECT priezvisko , meno  
FROM student  
ORDER BY priezvisko ;
```

```
SELECT priezvisko , meno  
FROM student  
ORDER BY priezvisko , meno  
DESC ;
```

Klauzuly

LIMIT, OFFSET

- *limit* - voliteľná klauzula príkazu *select*. Obmedzí počet riadkov, ktoré vráti dotaz.
- *offset* - definuje, koľko riadkov má dotaz vynechať pred začatím počítania riadkov definovaných v *limit*.
- Syntax:

```
SELECT select_list
  FROM table_expression
  [ ORDER BY ... ]
  [ LIMIT { number | ALL } ]
  [ OFFSET number ];
```

- Príklad:

```
SELECT priezvisko , meno
FROM student
ORDER BY priezvisko , meno
LIMIT 10 OFFSET 2;
```

Klauzuly

FETCH

- *fetch* - ekvivalent *limit*, avšak je súčasťou SQL štandardu (zavedený v SQL:2008).
- Syntax:

```
OFFSET start { ROW | ROWS }  
FETCH { FIRST | NEXT } [ row_count ]  
{ ROW | ROWS } ONLY
```

- Príklad:

```
SELECT priezvisko , meno  
FROM student  
ORDER BY priezvisko , meno  
OFFSET 5 ROWS  
FETCH FIRST 5 ROW ONLY;
```


NULL hodnota

NULL hodnota

- Špeciálna hodnota pre neznámu alebo nedefinovanú hodnotu.
- Špecifické správanie pri WHERE klauzule, agregračných funkciách, atď.
- Na vyhodnotenie pravdivosti SQL používa trojhodnotovú logiku:

NOT(A)		AND(A, B)					OR(A, B)				
A ¬A		A ∧ B		B			A ∨ B		B		
A	¬A			F	U	T	A		F	U	T
F	T	F	F	F	F	F	F	F	F	U	T
U	U	U	F	F	U	U	U	U	U	U	T
T	F	T	F	F	U	T	T	T	T	T	T

Alias

Alias

- Umožňuje dočasne premenovať stĺpec (kľúčové slovo **AS**) alebo tabuľku (bez kľúčového slova, niekedy sa označuje ako tabuľkové premenné).
- Používa sa napríklad na:
 - skrátený zápis (čitateľnosť),
 - používa sa na rozlíšenie stĺpcov s rovnakým menom,
 - "pekné" pomenovanie stĺpca.
- Príklad:

```
SELECT d.surname || ' ' || d.name AS meno  
FROM dbuser d;
```

Vytváranie entít pomocou SELECT

Vytvorenie pohľadu z výsledkov príkazu SELECT

- Pohľad je pomenovaný SELECT uložený v databáze.
- Virtuálna (odvodená) tabuľka.
- Príklad:

```
CREATE VIEW nazov_pohladu AS  
SELECT stlpce  
FROM tabulky  
WHERE podmienky;
```

Vytváranie entít pomocou SELECT

Vytvorenie novej tabuľky z výsledkov príkazu SELECT

- S takto vytvorenou tabuľkou je možné robiť všetky DML príkazy jazyka SQL.
- Úpravy v novej tabuľke sa nedotknú pôvodných údajov.
- Príklad:

```
CREATE TABLE názov AS  
SELECT stĺpce  
FROM tabuľky  
WHERE podmienky;
```

Vytváranie entít pomocou SELECT

SELECT v príkaze INSERT

- Vloží do tabuľky výsledok dopytu.
- Príklad:
INSERT INTO tabuľka
select príkaz;
- Príklad:
INSERT INTO follows
SELECT * FROM friendsWith;

Zdroje a použitá literatúra

- Jaroslav Porubän, Miroslav Biňas, Milan Nosál, *Databázové systémy*- prednášky, FEI, TUKE, 2011 - 2016.
- <https://www.postgresql.org/docs/>.
- <https://www.postgresqltutorial.com/>.
- Ramez Elmasri, Shamkant B. Navathe: *Fundamentals of Database Systems*, Addison Wesley, 5 edition, 2006, 1168 p. ISBN 0321369572.
- Abraham Silberschatz, Henry F. Korth, S. Sudarshan: *Database System Concepts*, The McGraw-Hill Companies, 2011, 6th edition, ISBN 978-0-07-352332-3.
- Ярцев В.П. *Організація баз даних та знань*: навчальний посібник, Державний університет телекомунікацій, 214с, 2018.

Otázky?